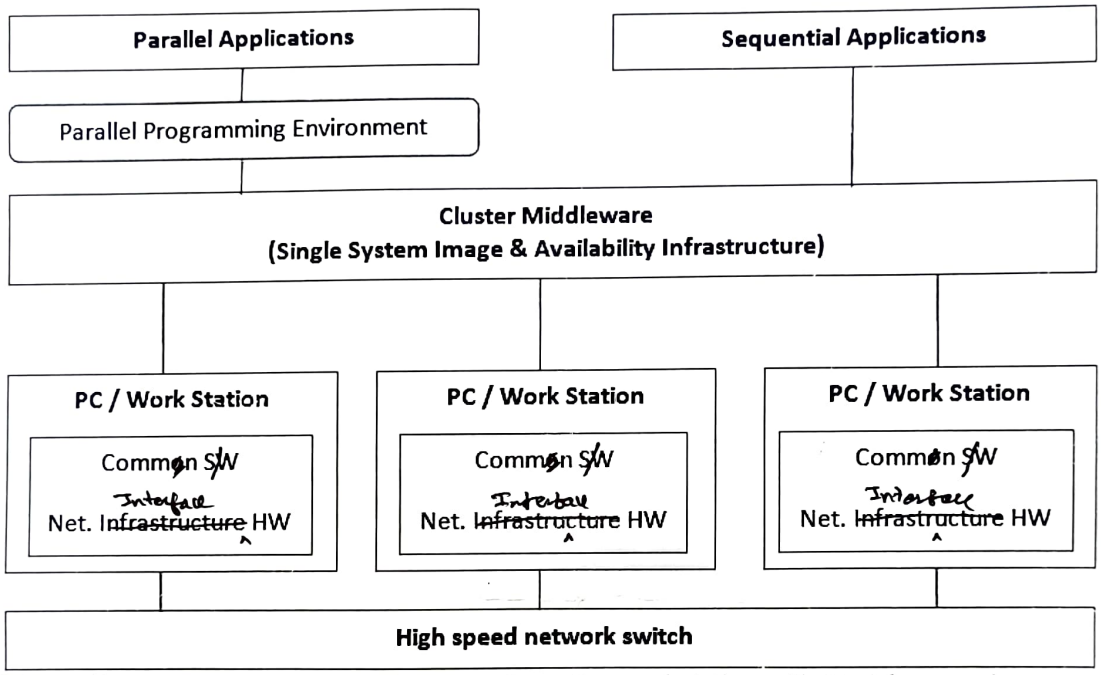
Q. Cluster Computer (M-5)

- A cluster is a type of parallel or distributed processing system.
- A computer cluster is a set of loosely or tightly connected computers that work together so that, in many respects, they can be viewed as a single system.
- It consists of a collection of interconnected stand-alone computers working together as a single, integrated computing resource connected via high speed interconnects.
- Unlike grid computers, computer clusters have each node set to perform the same task, controlled and scheduled by software.
- The components of a cluster are usually connected to each other through fast LAN, with each node running its own instance of an operating system.
- In most circumstances, all of the nodes use the same hardware and the same operating system, although in some setups different operating systems can be used on each computer, or different hardware.



- **High performance computers:** These can be PCs, workstations, SMP. These nodes generally have memory, I/O facility and operating system.
- **Modern operating system:** OS should be layered or micro kernel based. Typical OS used in clusters are Linux, Solaris, Windows NT.
- **High speed networks/Switches:** Interconnects as Gigabit Ethernet, ATM etc are suitable and high performance switches are used for connections.
- **Network Interface hardware** is generally various NICs which is an interface to network.
- **Communication Software** includes various fast communication protocols and services as AM, FM which is used for speedy communication between nodes.
- **Cluster Middleware:** its basic aim is to provide SSI and SAI services to the user. It includes
  - Hardware like Hardware DSM, Memory channel etc.
  - Operating system kernel also called the gluing layer like Solaris MC, GLUnix etc.
  - Applications and subsystem like Software DSM, File systems Management tools, RMS software etc.
- **Parallel Programming environment and Tools:** These provide portable, efficient and easy to use tools to work in parallel environment and run applications simultaneously. E.g. PVM, MPI etc.
- **Applications:** Sequential as well as parallel applications can be run on a cluster.

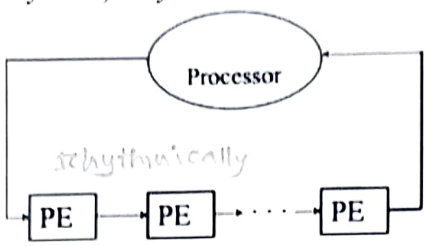
Blood pressure - 120/80 mm Hg → 120 → to systolic pressure, 80 → to diastolic pressure

• Like blood flow through different valve in heart, and each valve performs diff. work e.g. blood loops CO_2 and gain O_2.

Systolic architecture (5)

Basic Structure:

- A network of PEs that rhythmically produces and passes data through the system is called systolic architecture.
- A number of usually similar processing elements connected together to implement this architecture.
- Data move between PE's in a rhythmic fashion.
- Typically, fully pipelined. Also communication between neighboring PE's only.
- Some processors (especially boundary ones) may be different than the rest.



e.g. agile testing

- It is used as a co-processor in combination with a host computer and the behavior is analogous to the flow of blood through heart; thus named SYSTOLIC.

Working Principle:

→ pronounce

- Systolic architecture consists of simple cells arranged in some regular pattern (linear, bi-directional, triangular, hexagonal, etc.) where each cell usually performs one operation.
- Each processing cell is connected to its neighbor or to a neighborhood of processing elements by short signal paths.
- Both parallel and pipelined execution is implemented.
- A function that is to be performed can be represented by a set of Functional Primitives.
- For e.g. when we are using our simple architecture then we can perform at most five million operations per second whereas by using systolic arrays in systolic architecture we can operate the system at a speed of 30 million operations per second.

Example of systolic network

1. Linear network
2. Bi-dimensional network
3. Triangular
4. Hexagonal network

etc

Use:

These architectures are well suited for a VLSI or FPGA network implementation

Advantages and Disadvantages:

- ADVANTAGES

Faster

Scalable

- DISADVANTAGES

Expensive

Highly specialized for particular applications

Difficult to build

Neighbour - someone who lives very close to you: Have you met Pat, my next-door neighbour?
Neighbourhood - the area of a town that surrounds someone's home, or the people who live in this area: There were lots of kids in my neighbourhood when I was growing up.

Q. What do you mean by program flow mechanism? (Q-1)
Ans. A program flow is the order in which individual statements, instructions or function calls of a program are executed or evaluated.

Q. Compare between Control flow, data flow and demand-driven mechanism. (Q-4)
Ans.

Control flow	Data flow/data driven	Demand-driven
Conventional machines used control flow mechanism in which order of program execution explicitly stated in user programs.	Dataflow machines in which instructions can be executed by determining operand availability.	Demand driven (E.g. Reduction machine) mechanism trigger an instruction's execution based on the demand for its result.
Control-flow computers use shared memory to hold program instructions and data objects.	Instead of being stored in a shared memory, data are directly held inside instructions.	Memory is not shared.
The execution of one instruction may produce side effect on other instructions since memory is shared.	No side effect as memory is not shared.	No side effect as memory is not shared.
Have program counter to determine the next instruction to execute.	Do not have a **program counter** as the execution of instructions is solely determined based on the availability of input arguments to the instructions.	PC is not required.
It is a bottom-up approach.	It is a bottom-up approach.	It is a top-down approach.

- It dominate the market
Still in the research stage
Still in the research stage.

Dataflow computer architecture:

- Control-flow computers use shared memory to hold program instructions and data objects.
- Variables in the shared memory are updated by many instructions.
- The execution of one instruction may produce side effect on other instructions since memory is shared.
- Dataflow architecture is a computer architecture that directly contrasts the traditional von Neumann architecture or control flow architecture.
- Dataflow architectures do not have a **program counter** the execution of instructions is solely determined based on the availability of input arguments to the instructions, so that the order of instruction execution is unpredictable i.e. behavior is indeterministic.
- Instead of being stored in a shared memory, data are directly held inside instructions.
- Computational results (data tokens) are passed directly between instructions.
- The data driven scheme requires no shared memory, no PC and no control sequence.
- Although no commercially successful general-purpose computer hardware has used a dataflow architecture, it has been successfully implemented in specialized hardware such as in digital signal processing, network routing, graphics processing, telemetry, and more recently in data warehousing.
- Dataflow architectures that are deterministic in nature enable programmers to manage complex tasks such as processor load balancing, synchronization and accesses to common resources.
- In the control flow architecture, compilers analyze program source code for data dependencies between instructions in order to better organize the instruction sequences in the binary output files. The instructions are organized sequentially but the dependency information itself is not recorded in the binaries. Binaries compiled for a dataflow machine contain this dependency information.
- A dataflow compiler records these dependencies by creating unique tags for each dependency instead of using variable names. By giving each dependency a unique tag, it allows the non-dependent code segments in the binary to be executed out of order and in parallel. Compiler detects the loops, break statements and various programming control syntax for data flow.

A machine language program of a data flow computer is represented by **a data flow program graph.** This graph is made up of **operators** connected by **arcs.** Arcs carry **tokens** which have **values.** An operator with **two input acs** and **one output arc** is shown below in Fig.

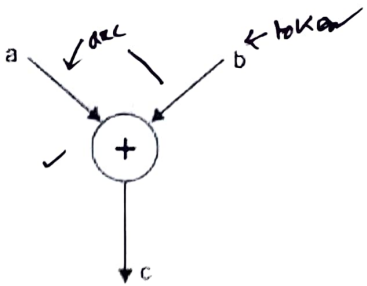


Fig. A data flow operator

The operator of Fig. is enabled only if tokens are present in both its input arcs and no token is present in its output arc. **Tokens are placed on and removed from the arcs according to the firing rules.** To explain the firing rule, let us say $a = 5$ and $b = 6$ in above Fig. Then, we have :

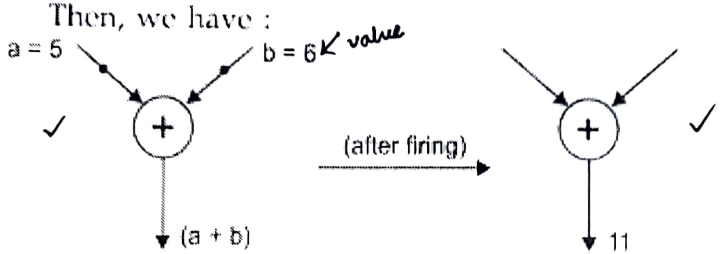


Fig. Firing rules

Please note that here, an operator fires, *i.e.*, the tokens in the two input arcs are removed, the operation (+) is applied on the values carried by the tokens and a token with the result value is placed on the output arc. This is what exactly happens in this example.

Q. Draw data flow graph to represent the following computations: (5)

1. A = P+Q
2. B = A/B
3. C = P*A
4. D = C-B
5. E = C*A
6. F = D/E

